

ANALYSE TECHNIQUE — StatEduc Mobile

Document d'analyse approfondie du code source de l'application mobile **stateduc_mobile**

Auteur : Abdoul Nasser Kailou

1. Vue d'ensemble

StatEduc Mobile est une application hybride Android de collecte de données pour le secteur éducatif en Afrique. Elle se connecte à un serveur StatEduc central pour télécharger des configurations de campagnes et des formulaires de saisie dynamiques. Une fois les données chargées, la saisie peut se faire en mode **hors ligne (offline)**. Les données collectées sont ensuite synchronisées avec le serveur.

2. Stack technique

2.1 Technologie principale

Composant	Technologie	Version
Framework mobile	Apache Cordova	<code>cordova-android</code> ^13.0.0
Langage UI	HTML5 + JavaScript (ES5)	—
UI Framework	jQuery Mobile	1.4.5
Bibliothèque DOM	jQuery	1.11.1
Plateforme cible	Android uniquement	API via Gradle
Gestionnaire de paquets	npm	—

2.2 Plugins Cordova utilisés

Plugin	Rôle
<code>cordova-plugin-dialogs</code> v2.0.2	Boîtes de dialogue natives (alert, confirm)
<code>cordova-plugin-network-information</code> v3.0.0	Détection de la connectivité réseau
<code>cordova-plugin-vibration</code> v3.1.1	Vibration du téléphone
<code>hazems-cordova-plugin-sms</code> v0.0.2	Envoi de SMS (passerelle de secours)

2.3 Bibliothèques JavaScript additionnelles

Bibliothèque	Usage
<code>jquery.mobile-1.4.5.min.js</code>	UI mobile (pages, popup, listview, selectmenu)
<code>jquery.ui.datepicker.js</code>	Sélecteur de dates
<code>jquery.mobile.datepicker.js</code>	Intégration datepicker / jQuery Mobile
<code>jquery.are-you-sure.js</code>	Détection des modifications non sauvegardées dans les formulaires
<code>ays-beforeunload-shim.js</code>	Shim beforeunload pour mobile
<code>jquery.mobile.dynamic.popup.js</code>	Popups dynamiques

3. Architecture

3.1 Structure des fichiers

```

stateduc_mobile/
├── config.xml           ← Configuration Cordova (id app, permissions, plugins)
├── package.json         ← Dépendances npm / Cordova
├── www/                 ← Code source de l'application (HTML/JS/CSS)
│   ├── index.html      ← Page principale (login, settings, liste campagnes)
│   ├── camp.html       ← Page campagne + établissements + saisie
│   ├── new_camp.html    ← Chargement d'une nouvelle campagne
│   ├── settings.html   ← Paramètres avancés (PIN, question sécurité)
│   ├── css/
│   │   ├── style.css   ← Styles personnalisés
│   │   └── themes/     ← Thème jQuery Mobile personnalisé "stateduc"
│   ├── img/            ← Logos, icônes
│   └── js/
│       ├── default.js   ← Modèles StmYear, StmFilter (gestion en localStorage)
│       ├── status.js    ← Modèle StmStatus (statuts établissements)
│       ├── systems.js   ← Modèle StmSystem (secteurs d'enseignement)
│       ├── campagnes.js ← Modèle StmCampagne, StmLocalisation (campagnes de collecte)
│       ├── regroups.js  ← Modèle StmRegroup (entités administratives hiérarchiques)
│       ├── etabs.js     ← Modèle StmEtab, StmCollectData (établissements + données)
│       ├── questions.js ← Modèle StmQuestion, StmRule (formulaires + règles validation)
│       ├── users.js     ← Gestion utilisateur (login/logout serveur)
│       ├── charge_camp.js ← Téléchargement campagne depuis serveur (AJAX)
│       ├── calc_total.js ← Calculs automatiques dans matrices 2D
│       ├── error_msg.js ← Messages d'erreur de validation
│       ├── script.js    ← Utilitaires (dialogs, connexion, navigation, ctrl_saisie)
│       ├── start.js     ← Initialisation au démarrage
│       ├── page_index.js ← Logique page accueil (PIN, settings, liste campagnes)
│       ├── page_new_camp.js ← Logique page chargement campagne
│       ├── page_camp.js  ← Logique page campagne (liste établissements, filtres)
│       ├── page_etab.js  ← Logique page établissement (saisie, sauvegarde, envoi)
│       └── page_settings.js ← Logique page paramètres (PIN, question sécurité)
├── platforms/android/   ← Code généré Cordova pour Android (Gradle/Java)
└── plugins/             ← Plugins Cordova installés

```

3.2 Navigation entre pages (multi-page SPA)

```

index.html
├── #p_accueil          → Authentification locale par PIN
├── #p_acc_settings     → Configuration URL serveur
├── #p_lst_camps        → Liste des campagnes chargées
│   └── camp.html
│       ├── #p_camp     → Détail campagne + filtre secteur + liste établissements
│       └── #p_etab     → Saisie des formulaires d'un établissement
new_camp.html
└── #p_new_camp         → Chargement d'une nouvelle campagne depuis le serveur
settings.html
└── #p_settings        → Paramètres (PIN, question sécurité)

```

4. Stockage des données

4.1 Type de stockage

L'application n'utilise **aucune base de données embarquée** (ni SQLite, ni IndexedDB).

Tout le stockage repose sur le **localStorage** et **sessionStorage** du navigateur WebView.

4.2 Clés localStorage (persistant)

Clé	Contenu
<code>stm_UrlServer</code>	URL du serveur StatEdu
<code>stm_UserPin</code>	PIN local de l'utilisateur (code d'accès)
<code>stm_UserSecQuestion</code>	Index de la question de sécurité
<code>stm_UserSecQuestionValue</code>	Réponse à la question de sécurité
<code>stm_CurrYear</code>	Année scolaire courante
<code>stm_FilterName</code>	Nom du filtre (ex: période)
<code>stm_CurrFilter</code>	Filtre actif courant
<code>stm_ChargedYears</code>	JSON – liste des années chargées
<code>stm_ChargedFilters</code>	JSON – liste des filtres disponibles
<code>stm_ChargedSystems</code>	JSON – liste des secteurs d'enseignement
<code>stm_ChargedStatus</code>	JSON – statuts établissements
<code>stm_ChargedCamps</code>	JSON – liste des campagnes chargées
<code>stm_ChargedRegroups</code>	JSON – entités administratives (région, département...)
<code>stm_ChargedTypeRegroups</code>	JSON – types d'entités administratives
<code>stm_ChargedEtabs</code>	JSON – liste des établissements
<code>stm_ChargedLocs_{id_camp}</code>	JSON – chaînes de localisation par campagne
<code>stm_ChargedQst_{id_camp}_{id_sys}</code>	JSON – liste des questionnaires par campagne/secteur
<code>stm_ChargedQstHtml_{id_camp}_{id_qst}_{id_sys}</code>	HTML brut du formulaire de saisie
<code>stm_ChargedThemeRule_{id_camp}_{id_qst}_{id_sys}</code>	JSON – règles de validation par formulaire
<code>stm_EtabCollectData_{id_etab}_{id_qst}[_{id_filter}]</code>	JSON – données collectées par établissement/formulaire
<code>stm_SmsGatewayNum</code>	Numéro passerelle SMS (optionnel)
<code>stm_SmsGatewayKey</code>	Clé passerelle SMS (optionnel)

4.3 Clés sessionStorage (session)

Clé	Contenu
<code>stm_userData</code>	JSON – données de l'utilisateur connecté
<code>stm_userPass</code>	Mot de passe en clair (session uniquement)
<code>stm_UserPin</code>	PIN validé pour la session courante

5. Communication serveur

5.1 Protocole

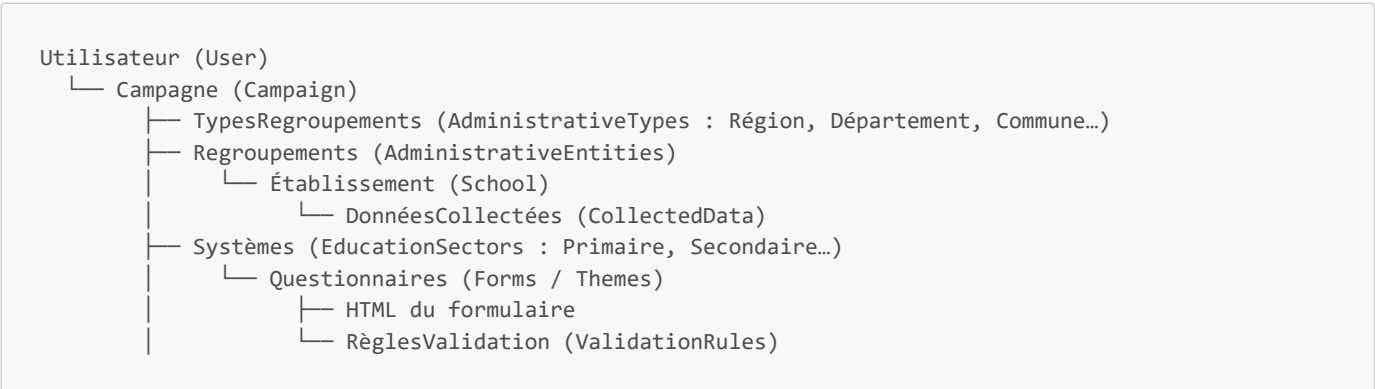
- **HTTP REST** (GET/POST) avec authentification **HTTP Basic** (Base64)
- Format de réponse : **JSON** (enveloppe { `se_status`: 200, `se_data`: [...] })
- Timeout : 10 000 ms (login) à 120 000 ms (envoi données)
- CORS activé côté config Cordova (`<access origin="*" />`, `usesCleartextTraffic="true"`)

5.2 Endpoints utilisés

Endpoint	Méthode	Description
/user_ident.php/user/{login}/{mdp}	GET	Authentification utilisateur
/user_ident.php/logout/xxxx/xxxx	GET	Déconnexion
/user_ident.php/user_test_login/	GET	Vérification session
/user_camp.php/new_camp/{id_user}/1	GET	Liste des campagnes disponibles
/user_camp.php/reg_camp/{login}/{id_camp}/1	GET	Entités administratives de la campagne
/user_camp.php/typ_reg_camp/{id_user}/{id_camp}/{typeregroups}	GET	Types d'entités admin.
/user_camp.php/etabs_status/	GET	Statuts des établissements
/user_camp.php/etabs_camp/{id_user}/{id_camp}/1	GET	Établissements de la campagne
/user_camp.php/locs_camp/{id_user}/{id_camp}	GET	Chaînes de localisation
/user_camp.php/sys_camp/{id_user}/{id_camp}	GET	Secteurs d'enseignement
/data_camp.php/theme_camp/{id_camp}/{id_sys}/eng	GET	Liste des questionnaires
/data_camp.php/html_theme_camp/{id_camp}/{id_qst}/eng	GET	HTML du formulaire
/data_camp.php/regle_theme_camp/{id_qst}/{id_sys}	GET	Règles de validation
/data_save.php/theme_save/{login}/{id_camp}/{id_sys}/{id_qst}/{id_etab}/{filter}	POST	Envoi données collectées
/data_reload.php/theme_data/{login}/{id_sys}/{id_qst}/{id_camp}/{id_etab}/{filter}	GET	Rechargement données du serveur

6. Modèle de données

6.1 Hiérarchie conceptuelle



```
├─ Localisations (LocationChains : lien Système ↔ Regroupements ↔ Établissements)
├─ Filtres (Filters : ex: Période)
```

6.2 Objets principaux JavaScript

Classe	Fichier	Description
<code>StmYear</code>	<code>default.js</code>	Année scolaire
<code>StmFilter</code>	<code>default.js</code>	Filtre (ex: période)
<code>StmSystem</code>	<code>systems.js</code>	Secteur d'enseignement
<code>StmStatus</code>	<code>status.js</code>	Statut d'établissement
<code>StmCampagne</code>	<code>campagnes.js</code>	Campagne de collecte
<code>StmLocalisation</code>	<code>campagnes.js</code>	Chaîne de localisation
<code>StmRegroup</code>	<code>regroups.js</code>	Entité administrative
<code>StmEtab</code>	<code>etabs.js</code>	Établissement scolaire
<code>StmCollectData</code>	<code>etabs.js</code>	Donnée collectée (clé/valeur/type)
<code>StmQuestion</code>	<code>questions.js</code>	Questionnaire/Formulaire
<code>StmRule</code>	<code>questions.js</code>	Règle de validation d'un champ

7. Fonctionnalités détaillées

7.1 Authentification locale (PIN)

- Lors du premier lancement, l'utilisateur crée un **PIN** + une question de sécurité
- Le PIN est stocké en **localStorage**
- À chaque session, le PIN est vérifié et stocké en **sessionStorage**
- Option "Mot de passe oublié" via la question de sécurité

7.2 Configuration serveur

- L'URL du serveur est saisie manuellement dans les paramètres
- Stockée en localStorage (`stm_UrlServer`)

7.3 Authentification serveur

- Login/mot de passe transmis via HTTP Basic Auth à chaque requête
- Session stockée en sessionStorage

7.4 Chargement d'une campagne (synchronisation descendante)

Processus séquentiel asynchrone (AJAX) :

1. Récupération des entités administratives (`reg_camp`)
2. Récupération des types d'entités (`typ_reg_camp`)
3. Récupération des statuts (`etabs_status`)
4. Récupération des établissements (`etabs_camp`)
5. Récupération des localisations (`locs_camp`)
6. Récupération des secteurs (`sys_camp`)
7. Pour chaque secteur → récupération des questionnaires (`theme_camp`)
8. Pour chaque questionnaire → récupération du HTML (`html_theme_camp`)
9. Pour chaque questionnaire → récupération des règles (`regle_theme_camp`)

7.5 Navigation dans une campagne

- Sélection du **secteur d'enseignement** → chargement des questionnaires
- Filtrage hiérarchique par **entités administratives** (drill-down : région → département → commune)
- Affichage de la liste des **établissements** correspondants
- Sélection d'un établissement → accès aux formulaires de saisie

7.6 Saisie de données (offline)

- Formulaires HTML dynamiques injectés dans la WebView
- Support de : `input[text]`, `input[radio]`, `input[checkbox]`, `select`
- Support des **grilles lignes** (tableaux dynamiques avec ajout/suppression de lignes)
- Support des **filtres** (ex: saisir des données par période)
- Sauvegarde locale en localStorage à chaque "Enregistrer"
- Détection des modifications non sauvegardées (plugin `are-you-sure`)
- Validation des données : type (int, decimal, date), taille, intervalle, champs obligatoires, énumérations

7.7 Envoi des données (synchronisation montante)

- Envoi par **POST HTTP** au serveur
- Possibilité d'envoyer : formulaire courant, tous les formulaires d'un établissement, ou tous les établissements d'une campagne
- Rechargement des données depuis le serveur (écrasement local)
- Envoi alternatif via **SMS** (passerelle SMS, découpage en morceaux de 148 caractères)

7.8 Calculs automatiques (calc_total.js)

- Calcul de totaux dans des matrices 2D (lignes × colonnes × mesures)
- Utilisé dans les formulaires HTML générés par le serveur

8. Diagnostic — Points forts et faiblesses

8.1 Points forts

Point	Description
Mode offline complet	Toutes les données nécessaires sont stockées localement
Formulaires dynamiques	Les écrans de saisie sont générés côté serveur → pas de mise à jour app nécessaire
Architecture modulaire JS	Chaque domaine fonctionnel a son propre fichier JS
Flexibilité des formulaires	Support de matrices, grilles, filtres périodiques

8.2 Points faibles / Problèmes détectés

A. Technologie obsolète et non maintenable

Problème	Sévérité
Apache Cordova est en fin de vie active de développement	 Critique
jQuery Mobile 1.4.5 (2014) n'est plus maintenu	 Critique
jQuery 1.11.1 (2014) — très ancienne version	 Critique
JavaScript ES5 sans modules ni transpilation	 Élevé
Pas de TypeScript ni de typage	 Élevé

B. Sécurité

Problème	Sévérité
Mot de passe stocké en clair dans <code>sessionStorage</code> (<code>stm_userPass</code>)	 Critique

Problème	Sévérité
PIN stocké en clair dans <code>localStorage</code> (<code>stm_UserPin</code>)	● Critique
<code>usesCleartextTraffic="true"</code> → HTTP non chiffré autorisé	● Critique
Authentification HTTP Basic (non sécurisée sans HTTPS)	● Élevé
Utilisation de <code>eval()</code> dans <code>ctrl_saisie</code> et <code>calc_total.js</code>	● Élevé

C. Qualité du code

Problème	Sévérité
Variables globales massives (pas d'encapsulation)	● Élevé
Pas de gestion centralisée des erreurs	● Élevé
Appels AJAX non chaînés / pas de gestion de concurrence	● Élevé
Bugs détectés : <code>stmDevice.isOnline()</code> retourne toujours <code>true</code> (code mort)	● Moyen
Bug : <code>StmData.getKey()</code> et <code>getValue()</code> affectent au lieu de retourner	● Moyen
<code>String.prototype.startsWith</code> polyfill inutile (ES6 standard maintenant)	● Faible

D. Architecture

Problème	Sévérité
Pas de base de données embarquée (<code>localStorage</code> saturé sur gros volumes)	● Élevé
<code>localStorage</code> limité à ~5 Mo sur Android WebView	● Élevé
Pas de vrai système de routing (multi-fichiers HTML + jQuery Mobile pages)	● Élevé
Pas de gestion offline robuste (pas de Service Worker, pas de SQLite)	● Élevé
Plateforme Android uniquement (pas d'iOS)	● Moyen
Formulaires HTML injectés bruts → XSS possible si serveur compromis	● Moyen

E. Expérience utilisateur

Problème	Sévérité
Interface jQuery Mobile très datée	● Élevé
Pas d'indicateur de progression lors du chargement long	● Moyen
Messages d'erreur par <code>alert()</code> natif	● Faible
Pas de responsive design moderne	● Moyen

9. Recommandations pour la réécriture Flutter

9.1 Remplacement des composants

Original (Cordova)	Équivalent Flutter
<code>localStorage</code>	<code>sqflite</code> (SQLite embarqué) + <code>shared_preferences</code>
<code>sessionStorage</code>	<code>Provider</code> / <code>Riverpod</code> (state en mémoire)
jQuery Mobile pages	Flutter <code>Navigator</code> + <code>MaterialPageRoute</code>
jQuery Mobile UI	Flutter Material Design widgets

Original (Cordova)	Équivalent Flutter
AJAX jQuery	<code>http</code> ou <code>dio</code> package
HTTP Basic Auth	<code>dio</code> avec intercepteurs
Formulaires HTML dynamiques	Parsing JSON → widgets Flutter dynamiques
<code>calc_total.js</code>	Dart fonctions pures
<code>cordova-plugin-network-information</code>	<code>connectivity_plus</code>
<code>cordova-plugin-dialogs</code>	<code>showDialog()</code> Flutter natif
<code>cordova-plugin-vibration</code>	<code>vibration</code> package
<code>hazems-cordova-plugin-sms</code>	<code>telephony</code> ou <code>flutter_sms</code>
<code>are-you-sure</code> jQuery plugin	<code>WillPopScope</code> + état dirty

9.2 Architecture Flutter recommandée

```

lib/
├── main.dart
├── config/           ← Configuration globale
├── models/           ← Modèles de données (Campaign, School, Form...)
├── services/
│   ├── api_service.dart ← Appels HTTP (dio)
│   ├── local_db_service.dart ← SQLite (sqflite)
│   └── auth_service.dart ← Authentification PIN + serveur
├── providers/        ← Gestion d'état (Riverpod/Provider)
├── screens/
│   ├── login/        ← Écran PIN
│   ├── settings/     ← Paramètres serveur
│   ├── campaigns/    ← Liste + chargement campagnes
│   ├── schools/       ← Navigation hiérarchique + liste établissements
│   └── data_entry/    ← Saisie dynamique formulaires
├── widgets/
│   ├── dynamic_form/ ← Widgets formulaire dynamique
│   └── common/        ← Widgets réutilisables
└── utils/             ← Utilitaires (validation, calculs, formatage)

```

9.3 Points d'attention critiques pour la réécriture

- Formulaires dynamiques** : Le challenge principal est de reproduire le rendu de formulaires HTML arbitraires (matrices, grilles, champs conditionnels) en widgets Flutter générés depuis JSON/HTML.
- Synchronisation offline** : Utiliser SQLite pour stocker des volumes importants de données.
- Calculs de totaux** : La logique de `calc_total.js` doit être reproduite en Dart.
- Grilles lignes** : Les tableaux avec ajout dynamique de lignes nécessitent une gestion d'état réactive.
- Sécurité** : Stocker les mots de passe avec `flutter_secure_storage`, activer HTTPS.

10. Résumé

Critère	Valeur
Langage	JavaScript ES5 (HTML/CSS)
Framework	Apache Cordova + jQuery Mobile 1.4.5
Plateforme	Android uniquement
Base de données	localStorage / sessionStorage (WebView)

Critère	Valeur
Communication	HTTP REST + HTTP Basic Auth
Mode offline	✅ Oui (localStorage)
Tests automatisés	❌ Aucun
Documentation code	❌ Minimale (quelques commentaires)
Maintenabilité	● Très faible (technologies obsolètes)
Sécurité	● Insuffisante (mots de passe en clair, HTTP)
Maturité du code	● Fonctionnel mais fragile